Underwriting Control — Documentation

Underwriting control — WixPayments-connected apps

Cloud App (no install needed)

The **App Compliance Screener** is live at: **app-compliance-screener-tihxaacrkagcy2ijutdvsk.streamlit.app**

Open the link, paste any app URL, and get an instant compliance verdict. No setup, no install, works from any browser. See `docs/TEAM-EMAIL-SCREENER.md` for a short tutorial.

What works on cloud: URL screening (fast + deep scrape), policy classification, findings table, review dropdowns, KPIs, filters, page content viewer.

What doesn't work on cloud: Trino queries (requires VPN/SSO). Use the local version below for Trino features.

Two tools for policy compliance review of WixPayments-connected Base44 apps:

Tool 1 — UW Lookup (`streamlit_uw.py`, port 8501): Look up any app by ID/MSID/WP account, view full profile + conversation + cached verdict, run the LLM underwriting pipeline.

Tool 2 — App Compliance Screener (`streamlit_screener.py`, port 8502): Paste URLs → instant rule-based policy classification → verdict. Batch-screen the full Trino population (240+ apps) with conversation summaries as extra signal. Catches auth-walled apps where the public page reveals nothing but the builder conversation reveals the real product (e.g. cannabis shop with a "fashion" description).

UW Pipeline (two-step): (1) **Middleman** — uses conversation summary + scraped app content to summarize intent, what is sold through the app, and what the end shopper gets. (2) **Policy comparison** — defines what is sold in detail, compares to policy, then outputs **Allowed**, **Restricted**, or **Not-allowed** with short reasoning and **non-compliant subcategories** (if any). Scraping is on by default (`--no-scrape` to disable).

Trino data: The Cursor AI automatically refreshes `data/trino_full_population.json` (240 apps with conversation summaries) at the start of each session if the data is older than 24 hours. See `.cursor/rules/trino-data-refresh.mdc`.

Phase 1: First successful LLM run

1. **Set OpenAI API key** (one of):
   - In terminal: `export OPENAI_API_KEY=sk-...` (use your real key), then run the script in the same shell.
   - Or create a `.env` file in the project root with one line: `OPENAI_API_KEY=sk-...`. The script loads it automatically (`.env` is in `.gitignore`; do not commit it).
2. **Install and run on sample data:**

   ```
   pip install -r requirements.txt
   python run_underwriting.py
   ```

   Defaults: `--apps data/sample_apps_from_trino.json`, `--policy policy/policy-excerpt.txt`, `--out output`. Confirm `output/` gets a conclusion file (e.g. `conclusion_6990c0d750b91ac92a28b8de.md`). Content will reflect placeholder policy until Phase 2.

Options: `--skip-llm` (dry run, no API call); `--model gpt-4o`; `--delay 1` (seconds after each LLM call to reduce rate limits).

Phase 2: Real policy in the pipeline

1. **Copy policy excerpts into `policy/policy-excerpt.txt`:**
   - Open *Copy of [Wix] Stripe Supportability Handling Guide_Jan25 (2).docx* (from project folder or your doc store).
   - Copy the sections that define: prohibited/restricted categories, supportability criteria, required disclosures, and any subcategories you want cited in conclusions.
   - Paste into `policy/policy-excerpt.txt` and save. No code changes required.
2. **Re-run on sample:** `python run_underwriting.py`. Spot-check the new conclusion: it should reference real policy categories/subcategories.

Phase 3: Full TRINO run

1. **Export TRINO results to JSON:**
   - Execute the canonical query in `docs/trino-query.sql` in your TRINO environment (no `LIMIT` for full set).

- Export the result set to a JSON file. Required fields per row: `app_id`, `app_name`, `app_url`, `conversation_summary`. Column names must match (e.g. `conversation_summary` not `conversation summary`).
- Save in the project, e.g. `data/apps_from_trino_20260222.json`.

2. **Run pipeline on the export:**

```
python run_underwriting.py --apps data/
        apps_from_trino_20260222.json --out output
```

   To keep this run's conclusions separate and get a manifest: `--run-id 20260222` (writes to `output/run_20260222/` and `manifest.json`). Use `--delay 1` for large runs to reduce rate limit risk.

3. The script retries on rate-limit errors (429) and waits `--delay` seconds after each LLM call by default.

Production test (7 apps)

A production test set is in `data/production_test_apps.json` (app_ids: 69990440a6ff02254b9a9862, 699856382da00d64831a5638, 6997d3f4d9d567adc5ed4020, 698406273ade17b9bd851188, 698e9adbbe8b990f1f47f603, 6996044001a264b9bcb16852, 69948763ca2ad1934bdd9654). Data was fetched from TRINO. To run the full pipeline (with LLM and optional scrape):

```
export OPENAI_API_KEY=sk-...   # or use .env
python run_underwriting.py --apps data/production_test_apps.json
        --out output --run-id production_test_20260222 --scrape
```

Conclusions will be written to `output/run_production_test_20260222/` with a `manifest.json`. Dry run (no API key): add `--skip-llm`.

Phase 4: Optional enhancements

- **Scrape app URLs:** Add `--scrape` to fetch each app's URL and include landing-page text as evidence in the conclusion. Requires the app to be publicly reachable.
- **Run versioning:** Use `--run-id <id>` so conclusions go to `output/run_<id>/` and a `manifest.json` (run_id, started_at, finished_at, app_ids) is written. Lets you compare runs and see what changed.
- **Policy from .docx:** You can pass the Word policy file directly: `--policy "path/to/Copy of [Wix] Stripe Supportability Handling Guide_Jan25 (2).docx"`. The script uses `python-docx` to extract paragraph text (install: `pip install python-docx`).

Full profiles (dashboard profile + conversation)

The dashboard shows extended profile (user_description, public_settings, categories) and conversation history from `data/full_profiles.json`.

If you have direct Trino: run `python3 scripts/fetch_full_profiles_from_trino.py` once to populate it.

If you only have Trino MCP (no direct Trino): run MCP queries in Cursor (see `docs/trino-query-earliest-conversation-preview.sql` and others) to fetch earliest conversation preview, user logs, conversations, and app metadata; save results to `data/trino_earliest_conversation_preview.json`, `data/trino_user_logs.json`, `data/trino_conversations.json`, `data/trino_app_metadata.json` (format: `{rows: [[...]], col_names: [...]}`). Then run:

```
python3 scripts/write_trino_metadata_from_mcp.py   # writes app metadata (16 apps with content)
python3 scripts/build_full_profiles_from_trino.py  # merges into full_profiles.json
```

Project layout

- `.cursor/rules/underwriting.mdc` — project rule (output = written conclusion, structure, data sources)
- `.cursor/rules/trino-data-refresh.mdc` — auto-refresh Trino population at session start
- `docs/BASE44-UW-PROJECT-DOCUMENTATION.md` — full technical documentation (v3.0)
- `docs/trino-query.sql` — canonical TRINO query (WixPayments apps + conversation summary for underwriting)
- `prompts/underwriting-conclusion.md` — LLM prompt used every run
- `policy/policy-excerpt.txt` — policy text (replace with real excerpts from the .docx)
- `run_underwriting.py` — UW pipeline script
- `streamlit_uw.py` — UW Lookup dashboard (port 8501)
- `streamlit_screener.py` — App Compliance Screener (port 8502)
- `uw_app/app_screener.py` — URL → scrape + classify → verdict
- `uw_app/policy_classifier.py` — rule-based Stripe policy taxonomy
- `scripts/screen_with_trino.py` — batch-screen full Trino population
- `scripts/validate_screener.py` — validate accuracy vs known non-compliant apps
- `data/trino_full_population.json` — live Trino population (240 apps, auto-refreshed)
- `output/` — conclusion memos and screener results